

High-Performance Sequential Quasi-Monte Carlo for Football Match Modelling

From low-discrepancy filtering to Rao-Blackwellised sports prediction

Dissertation working draft

1 September 2026

Abstract

Sequential Monte Carlo (SMC) is a flexible method for online inference in nonlinear and non-Gaussian state-space models, but its accuracy is limited by the ordinary Monte Carlo scale. Sequential quasi-Monte Carlo (SQMC) replaces independent simulation inputs with randomised low-discrepancy point sets and preserves their regularity through Hilbert-ordered resampling. This document develops a single statistical and computational narrative for SQMC. It first derives the SMC, QMC and SQMC recursions and then maps SQMC to an accelerator implementation in JAX, including Sobol generation, Hilbert indexing, ordered inverse-CDF resampling and deterministic propagation. The same implementation is then applied to international football. Team attack and defence strengths evolve as a correlated Gaussian Ornstein-Uhlenbeck process, while match scores follow a bivariate Poisson observation model. Although the full state has two coordinates per team, each match observes only two teams. Rao-Blackwellisation exploits this sparse likelihood: only four coordinates are sampled with SQMC, while the remaining teams are updated analytically by Gaussian conditioning. The resulting RB-SQMC filter links the main limitation of QMC - effective dimension - to the structure of the football application, and provides an accelerator-ready route from historical results to filtered team strengths, score probabilities and tournament predictions.

Contents

1	Introduction	2
2	Sequential Monte Carlo, QMC and SQMC	3
2.1	State-space filtering	3
2.2	Sequential Monte Carlo	4
2.2.1	Algorithm 1: bootstrap particle filter	4
2.3	Quasi-Monte Carlo and randomisation	5
2.4	Sequential quasi-Monte Carlo	6
2.4.1	Algorithm 2: Hilbert-ordered SQMC	6
2.5	Effective dimension and long horizons	6
3	Accelerator implementation of SQMC	7
3.1	Why the operations fit accelerators	7
3.2	JAX software architecture	8
3.3	Sobol generation and scrambling	8

3.4	Generalised Hilbert sorting	8
3.5	Ordered resampling on device	9
3.6	Precision and performance boundaries	9
4	Modelling football matches with RB-SQMC	9
4.1	Sparse observations in a correlated state	9
4.2	Correlated attack-defence dynamics	10
4.3	Bivariate Poisson score likelihood	10
4.4	Rao-Blackwellisation	11
4.5	Rao-Blackwellised SQMC	12
4.5.1	Algorithm 3: RB-SQMC match update	12
4.6	From filtered strengths to match probabilities	13
4.7	Parameter estimation	13
5	Unified empirical evaluation	14
5.1	Statistical and computational questions	14
5.2	Accuracy and time-to-solution	14
5.3	Throughput and kernel timing	14
5.4	Football predictive evaluation	14
5.5	Dimension and long-horizon experiments	15
6	Discussion and conclusion	15
7	References	16

1 Introduction

Many forecasting problems share the same computational structure. A latent state changes over time, observations arrive sequentially, and the inferential target must be updated before the next observation becomes available. In football, the latent state may describe the attack and defence strengths of every team, while the observations are match scores. The state is high dimensional because a competition contains many teams, but an individual match depends directly on only two of them. This combination - a large evolving state and a sparse local observation - motivates both the statistical model and the computational method developed in this document.

Sequential Monte Carlo (SMC), or particle filtering, is a natural starting point. It represents the filtering distribution by weighted particles and can accommodate nonlinear observation models such as a bivariate Poisson score likelihood. Its error at a fixed time is ordinarily of order $O_P(N^{-1/2})$, however, so a substantial reduction in error requires a large increase in the number of particles. Sequential quasi-Monte Carlo (SQMC) addresses this limitation by replacing independent uniforms with randomised low-discrepancy points. The challenge is resampling: a particle filter produces an unordered weighted cloud, whereas QMC relies on a regular mapping from the unit cube to the target distribution. SQMC resolves this conflict by sorting the particles along a Hilbert curve, applying inverse-CDF resampling to the first QMC coordinate and using the remaining coordinates for propagation [Gerber and Chopin \(2015\)](#).

SQMC introduces additional work. Point generation, Hilbert-key calculation, sorting, cumulative sums and inverse-CDF search all occur at every observation. The central computational proposition of this work is that these operations are well matched to modern accelerators when expressed as

regular array programs. Digital-net generation consists largely of integer masks, shifts and exclusive-or operations. Hilbert keys are computed by bit transposition and Gray-code traversal. Propagation and likelihood evaluation are batched over particles. Sorting and prefix scans are collective rather than embarrassingly parallel, but mature GPU primitives exist for both. JAX makes it possible to compose the complete filter, compile it through XLA and execute the same source on a CPU, GPU or TPU.

The football application completes the argument. A direct SQMC filter over K teams has state dimension $2K$, which is precisely the setting in which low-discrepancy constructions may lose their advantage. Yet one match observes only four latent coordinates: attack and defence for the two playing teams. Because the state transition is linear Gaussian, the other coordinates can be integrated out conditionally. Rao-Blackwellisation therefore reduces the sampled dimension from $2K$ to four. Each SQMC point has five coordinates: one for ancestor selection and four for propagation. The complete particle still retains a mean for every team, so information can travel through the correlated state covariance even when a team does not play.

This document makes four linked contributions:

1. it presents SMC, QMC and SQMC in a common state-space notation, including complete algorithms and the assumptions behind their error statements;
2. it maps the mathematical SQMC primitives to accelerator operations and to the JAX implementation in this repository;
3. it formulates a correlated dynamic attack-defence model with a sparse bivariate Poisson likelihood; and
4. it derives a Rao-Blackwellised SQMC filter that uses the four match-specific coordinates for QMC ordering and propagation while updating all remaining teams analytically.

The narrative proceeds from general filtering theory to hardware and then to the application. This order is intentional. The football model is not a separate example attached to an implementation chapter: its sparse likelihood provides the dimension reduction that makes SQMC statistically attractive, while the accelerator implementation makes the resulting filter practical at the particle counts required for prediction and parameter learning.

2 Sequential Monte Carlo, QMC and SQMC

2.1 State-space filtering

Let $X_t \in \mathcal{X} \subseteq \mathbb{R}^d$ be a latent Markov state and let Y_t be its observation. With initial density μ_0 , transition density f_t and observation density g_t , the joint model is

$$p(x_{0:T}, y_{0:T}) = \mu_0(x_0) g_0(y_0 | x_0) \prod_{t=1}^T f_t(x_t | x_{t-1}) g_t(y_t | x_t).$$

The filtering distribution $\pi_t(dx_t) = p(x_t | y_{0:t}) dx_t$ is obtained from the prediction-correction recursion

$$\pi_{t|t-1}(dx_t) = \int f_t(x_t | x_{t-1}) \pi_{t-1}(dx_{t-1}) dx_t,$$

$$\pi_t(dx_t) = \frac{g_t(y_t | x_t) \pi_{t|t-1}(dx_t)}{\int g_t(y_t | z) \pi_{t|t-1}(dz)}.$$

These integrals are available analytically for a linear Gaussian state-space model, but not when the likelihood is nonlinear or non-Gaussian. Particle filters replace them by empirical measures.

2.2 Sequential Monte Carlo

An SMC approximation is

$$\pi_t^N(dx) = \sum_{n=1}^N W_t^n \delta_{X_t^n}(dx), \quad W_t^n = \frac{w_t^n}{\sum_{m=1}^N w_t^m},$$

and estimates an expectation by

$$\pi_t^N(\varphi) = \sum_{n=1}^N W_t^n \varphi(X_t^n).$$

For a proposal $m_t(x_{t-1}, dx_t)$, an ancestor A_{t-1}^n is selected using $W_{t-1}^{1:N}$ and the new state is drawn from $m_t(X_{t-1}^{A_{t-1}^n}, \cdot)$. Its incremental weight is

$$w_t^n = \frac{g_t(y_t | X_t^n) f_t(X_t^n | X_{t-1}^{A_{t-1}^n})}{m_t(X_t^n | X_{t-1}^{A_{t-1}^n})}.$$

The bootstrap filter takes $m_t = f_t$, leaving $w_t^n = g_t(y_t | X_t^n)$. Degeneracy may be diagnosed by the effective sample size

$$\text{ESS}_t = \left\{ \sum_{n=1}^N (W_t^n)^2 \right\}^{-1}.$$

2.2.1 Algorithm 1: bootstrap particle filter

Input: N , observations $y_{0:T}$, simulators Γ_0 and Γ_t .

1. Draw independent $U_0^n \sim \mathcal{U}([0, 1]^d)$ and set $X_0^n = \Gamma_0(U_0^n)$.
2. Evaluate $w_0^n = g_0(y_0 | X_0^n)$ and normalise the weights.
3. For $t = 1, \dots, T$:
 - draw $A_{t-1}^n \sim \text{Categorical}(W_{t-1}^{1:N})$;
 - draw independent $V_t^n \sim \mathcal{U}([0, 1]^d)$;
 - set $X_t^n = \Gamma_t(X_{t-1}^{A_{t-1}^n}, V_t^n)$; and
 - evaluate $w_t^n = g_t(y_t | X_t^n)$ and normalise.
4. Return the weighted particle systems.

For fixed t and under standard regularity conditions, SMC errors are $O_P(N^{-1/2})$. The proportionality constant depends on the model, time, proposal and resampling scheme. SMC has $O(N)$ work per update with a linear-time resampler, but accuracy improves only with the square root of N .

2.3 Quasi-Monte Carlo and randomisation

For

$$I(\varphi) = \int_{[0,1]^s} \varphi(u) du, \quad \hat{I}_N(\varphi) = \frac{1}{N} \sum_{n=1}^N \varphi(u^n),$$

QMC selects $u^{1:N}$ to fill the unit cube more evenly than independent random points. Its star discrepancy is

$$D_N^*(u^{1:N}) = \sup_{a \in [0,1]^s} \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}\{u^n \in [0, a)\} - \prod_{j=1}^s a_j \right|.$$

For a function of bounded Hardy-Krause variation, Koksma-Hlawka gives

$$|\hat{I}_N(\varphi) - I(\varphi)| \leq V_{\text{HK}}(\varphi) D_N^*(u^{1:N}).$$

Well-constructed sequences can satisfy $D_N^* = O\{N^{-1}(\log N)^s\}$. This is a worst-case bound rather than a guarantee of $O(N^{-1})$ error for every integrand. Its dependence on s is also the source of the QMC dimension problem.

The Halton sequence uses radical inverses. If $n = \sum_{k \geq 0} a_k(n)b^k$, then

$$\phi_b(n) = \sum_{k \geq 0} a_k(n)b^{-k-1}, \quad u^n = (\phi_{b_1}(n), \dots, \phi_{b_s}(n)).$$

Sobol points are digital nets in base two. If $a(n)$ is the binary digit vector of n and C_j is the generator matrix for coordinate j , then

$$z_j = C_j a(n) \pmod{2}, \quad u_j^n = \sum_{r=1}^m z_{j,r} 2^{-r}.$$

The multiplication is over $\mathbb{F}_2$ and becomes bit masks and XOR in software. Randomised QMC applies a scramble that preserves low discrepancy while making each point marginally uniform. Consequently,

$$\mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N \varphi(\tilde{u}^n) \right] = I(\varphi),$$

and independent scrambles provide replicates for error estimation.

2.4 Sequential quasi-Monte Carlo

SQMC cannot simply substitute QMC uniforms into an ordinary particle filter, because categorical resampling destroys the ordering needed by a low-discrepancy transform. Let $\psi : \mathcal{X} \rightarrow [0, 1)^d$ be a component-wise monotone map and let $h : [0, 1)^d \rightarrow [0, 1)$ be a pseudo-inverse of a Hilbert curve. Each particle obtains the scalar key

$$z_{t-1}^n = h\{\psi(X_{t-1}^n)\}.$$

Let σ sort particles by this key. Generate an RQMC design $(u_t^n, v_t^n) \in [0, 1) \times [0, 1)^d$ and let τ sort the complete rows by their first coordinate. For the ordered weights, form

$$C_j = \sum_{k=1}^j W_{t-1}^{\sigma(k)}, \quad a_t^n = \min\{j : C_j \geq u_t^{\tau(n)}\}.$$

Propagation is the deterministic transform

$$X_t^n = \Gamma_t \left(X_{t-1}^{\sigma(a_t^n)}, v_t^{\tau(n)} \right).$$

The complete QMC row must remain paired: $u_t^{\tau(n)}$ selects the ancestor and $v_t^{\tau(n)}$ propagates that same design point.

2.4.1 Algorithm 2: Hilbert-ordered SQMC

1. Generate $u_0^{1:N} \subset [0, 1)^d$; compute $X_0^n = \Gamma_0(u_0^n)$ and normalised initial weights.
2. For $t = 1, \dots, T$:
 - generate $(u_t^n, v_t^n)_{n=1}^N \subset [0, 1)^{d+1}$;
 - Hilbert-sort the previous particle cloud to obtain σ ;
 - sort complete QMC rows by u_t^n to obtain τ ;
 - compute the CDF of $W_{t-1}^{\sigma(1:N)}$;
 - select ancestors by inverse transform using $u_t^{\tau(1:N)}$;
 - propagate with $v_t^{\tau(1:N)}$; and
 - evaluate and normalise the new potential weights.
3. Return the weighted particle systems.

The two sorts make the generic cost $O(N \log N)$ per update. Gerber and Chopin (2015) establish consistency and RQMC results that improve on ordinary Monte Carlo under their assumptions. Faster rates obtained for particular smooth RQMC integrations should not automatically be attributed to every SQMC model; the practical gain must be assessed for the target filter.

2.5 Effective dimension and long horizons

At a non-initial time, standard SQMC needs one coordinate for resampling and d coordinates for propagation. The discrepancy bound degrades with d , and the integrand may depend strongly

on many coordinates. Effective-dimension reduction is therefore central to SQMC design. Conditional simulation, Brownian bridges, principal components and Rao-Blackwellisation all aim to concentrate variation in a smaller set of QMC coordinates [Chopin and Gerber \(2017\)](#).

A separate result concerns time stability. In the particular one-dimensional linear Gaussian model analysed by [Gerber \(2026\)](#), a fixed-size particle filter has errors above a threshold infinitely often almost surely, while the analysed SQMC construction satisfies

$$\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\|_K = 0 \quad \text{almost surely.}$$

This provides motivation for long-horizon experiments but is not a universal time-uniform theorem for every SQMC application.

3 Accelerator implementation of SQMC

3.1 Why the operations fit accelerators

SQMC is a composition of regular array operations. Some are independent over particles; others require device-wide communication.

Statistical operation	Accelerator representation	Role in the filter
Sobol generation	masks, shifts, XOR and table reads	create paired (u, v) rows
State mapping	vectorised standardisation and sigmoid	map particles to the unit cube
Hilbert key	bit transpose and Gray-code scan	locality-preserving scalar order
Particle/QMC sorting	device sort	align ordered weights and QMC rows
Weight normalisation	log-sum-exp reduction	stable probabilities
Cumulative weights	parallel prefix scan	ordered discrete CDF
Ancestor selection	vectorised search	inverse transform at each u
Propagation	batched linear algebra	transform v to new states
Weighting	vectorised likelihood	assimilate the observation

Hilbert sorting does **not** eliminate the cumulative-weight scan. Standard SQMC still constructs and inverts the ordered CDF. The Hilbert key solves the multidimensional ordering problem; the prefix scan solves cumulative resampling. Both must be included in complexity and timing claims.

The graphics literature supplies useful implementation evidence. Integer-first radical inversion and Sobol generation avoid repeated floating-point accumulation, while Hilbert enumeration places nearby spatial locations close in a scalar order [Keller, Wachter and Binder \(2022\)](#); [Keller, Wachter and Binder \(2023\)](#). Rendering and filtering are different applications, but the low-level algebra is shared.

3.2 JAX software architecture

The implementation is separated into three layers:

- `sqmc/qmc/qmc.py` implements vectorised Halton and Sobol designs;
- `sqmc/hilbert_sort/hilbert_sort.py` computes packed Hilbert keys and stable sort permutations; and
- `rbsqmc/src/model/model_rbsqmc.py` combines RQMC generation, ordered resampling, Gaussian propagation and match weighting.

JAX traces the array program and lowers it through XLA. The outer time loop and inner same-day match loop use `jax.lax.scan`; particle likelihoods use `jax.vmap`; and the complete filter is compiled with `jax.jit`. Backend changes do not require separate CPU and GPU algorithms, although device-specific kernel fusion and floating-point reductions may differ.

3.3 Sobol generation and scrambling

The local Sobol generator uses checked-in Joe-Kuo direction integers. A left linear matrix scramble and digital shift produce

$$z_j = L_j C_j a + s_j \pmod{2},$$

where L_j is unit lower triangular and hence invertible over $\mathbb{F}_2$. The filter requests one independently scrambled design for each valid match update:

```
rqmc_points = generate_rqmc_points(
    key=sobol_key,
    n=n_particles,
    d=1 + effective_dimension,
)
```

For the football filter, `effective_dimension` is four. Column zero is used for ancestor selection and columns one to four drive the Gaussian propagation of the two teams.

3.4 Generalised Hilbert sorting

Continuous particle coordinates are standardised component-wise and mapped through a logistic transform,

$$\psi_j(x_j^n) = \left[1 + \exp \left(-\frac{x_j^n - \bar{x}_j}{s_j} \right) \right]^{-1}.$$

For $d \geq 2$, the implementation packs one key into 62 bits. The resolution per coordinate is

$$b_d = \left\lfloor \frac{62}{d} \right\rfloor, \quad q_j = \min \{ 2^{b_d} - 1, \lfloor 2^{b_d} \psi_j(x_j) \rfloor \}.$$

Coordinate bits are transposed into level-wise chunks. A fixed scan tracks the orientation of each Hilbert sub-cube, applies Gray-code travel and packs the decoded steps into a `uint64` key. Stable

`argsort` returns the required particle permutation. In one dimension, the implementation reduces to an ordinary stable sort.

3.5 Ordered resampling on device

The implementation follows the mathematical construction explicitly:

```
sort_idx = hilbert_sort(x_flat)
w = jnp.exp(log_weights - logsumexp(log_weights))
cdf = jnp.cumsum(w[sort_idx])

qmc_order = jnp.argsort(rqmc_points[:, 0], stable=True)
rqmc_sorted = rqmc_points[qmc_order]
positions = jnp.searchsorted(cdf, rqmc_sorted[:, 0])
ancestors = sort_idx[jnp.minimum(positions, n_particles - 1)]
v_t = rqmc_sorted[:, 1:]
```

Weights and particles share the same Hilbert permutation, and complete RQMC rows share the same u permutation. The selected ordered index is mapped back to the original particle array before gathering the complete ancestor.

3.6 Precision and performance boundaries

Digital point generation is primarily integer arithmetic, but weight normalisation and the cumulative CDF are floating-point reductions. The code enables 64-bit JAX arithmetic, uses `logsumexp` and clips RQMC endpoints before the Gaussian quantile. Reduced precision should be treated as an explicit experiment: parallel resampling work has documented loss of numerical accuracy in large single-precision cumulative sums [Murray, Lee and Jacob \(2016\)](#).

One SQMC update can be decomposed as

$$t_{\text{step}} = t_{\text{qmc}} + t_{\text{map}} + t_{\text{key}} + t_{\text{sort}} + t_{\text{scan}} + t_{\text{search}} + t_{\text{prop}} + t_{\text{weight}}.$$

This decomposition separates the particle-parallel work from the collective sort and scan and will be used in the empirical evaluation.

4 Modelling football matches with RB-SQMC

4.1 Sparse observations in a correlated state

Let there be K teams. Team k has latent attack and defence strength

$$x_t^{(k)} = \begin{pmatrix} x_t^{\text{att},k} \\ x_t^{\text{def},k} \end{pmatrix} \in \mathbb{R}^2, \quad x_t = \text{vec}\{x_t^{(1:K)}\} \in \mathbb{R}^{2K}.$$

Match t involves teams i_t and j_t , so its observation depends only on

$$\mathcal{O}_t = \{i_t, j_t\}, \quad x_t^{\mathcal{O}_t} \in \mathbb{R}^4.$$

This resembles a factorial state-space model because the likelihood is local, but the model below permits correlations between teams. When the between-team covariance is non-diagonal, the team processes are not independent; the model is more accurately described as a correlated Gaussian state-space model with a sparse likelihood. That distinction matters because a match can update the conditional beliefs about non-playing teams through covariance.

4.2 Correlated attack-defence dynamics

The initial state is

$$x_0 \sim \mathcal{N}_{2K}(\mu_0, \Sigma_0), \quad \Sigma_0 = \Gamma_0 \otimes B,$$

where $\Gamma_0 \in \mathbb{R}^{K \times K}$ describes between-team covariance and $B \in \mathbb{R}^{2 \times 2}$ describes the common attack-defence covariance. We require $\Gamma_0 \succ 0$ and $B \succ 0$. The constraint $\det(B) = 1$ removes the scale ambiguity $(c\Gamma_0) \otimes (B/c) = \Gamma_0 \otimes B$.

Team strengths evolve by a stationary Ornstein-Uhlenbeck transition. If Δt_t is the number of days since the preceding latent-state update,

$$\phi_t = \exp(-\kappa \Delta t_t),$$

$$x_t \mid x_{t-1} \sim \mathcal{N}(\mu_0 + \phi_t(x_{t-1} - \mu_0), (1 - \phi_t^2)\Sigma_0).$$

The parameter $\kappa > 0$ controls mean reversion. In the absence of matches, the predictive distribution relaxes towards the long-run $\mathcal{N}(\mu_0, \Sigma_0)$ distribution. The time increment permits irregular match schedules without treating a day, week and month as equivalent.

4.3 Bivariate Poisson score likelihood

Let the observed score be $y_t = (y_{t,i}, y_{t,j})$. Define

$$\lambda_{1,t} = \exp(\alpha + x_t^{\text{att},i_t} - x_t^{\text{def},j_t}),$$

$$\lambda_{2,t} = \exp(\alpha + x_t^{\text{att},j_t} - x_t^{\text{def},i_t}), \quad \lambda_3 = \exp(\beta).$$

Here α is a common log scoring-rate intercept; it is not a home advantage because it enters both intensities symmetrically. The shared Poisson component λ_3 induces dependence between the two scores. The bivariate Poisson likelihood is

$$\begin{aligned} G_t(y_t \mid x_t^{\mathcal{O}_t}) &= e^{-(\lambda_{1,t} + \lambda_{2,t} + \lambda_3)} \frac{\lambda_{1,t}^{y_{t,i}} \lambda_{2,t}^{y_{t,j}}}{y_{t,i}! y_{t,j}!} \\ &\times \sum_{r=0}^{\min(y_{t,i}, y_{t,j})} \binom{y_{t,i}}{r} \binom{y_{t,j}}{r} r! \left(\frac{\lambda_3}{\lambda_{1,t} \lambda_{2,t}} \right)^r. \end{aligned}$$

The likelihood is nonlinear and non-Gaussian, so the full filter is not a Kalman filter. Its dependence on only four coordinates nevertheless creates a conditionally Gaussian structure that can be Rao-Blackwellised [Karlis and Ntzoufras \(2003\)](#).

4.4 Rao-Blackwellisation

Partition the state into the playing coordinates $\mathcal{O}_t$ and remaining coordinates $\mathcal{R}_t$. Since the likelihood depends only on $x_t^{\mathcal{O}_t}$,

$$p(x_{0:T} | y_{1:T}) = p(x_{0:T}^{\mathcal{O}} | y_{1:T})p(x_{0:T}^{\mathcal{R}} | x_{0:T}^{\mathcal{O}}),$$

where $\mathcal{O}$ denotes the time-varying sequence of playing-team blocks. The first factor is approximated by particles; the second is recovered by Gaussian conditioning.

For component n , the predictive mean and shared covariance are

$$\mu_{t|t-1}^n = \mu_0 + \phi_t(\mu_{t-1}^n - \mu_0),$$

$$\Sigma_{t|t-1} = \phi_t^2 \Sigma_{t-1} + (1 - \phi_t^2) \Sigma_0.$$

Only the marginal

$$x_t^{\mathcal{O}_t, n} \sim \mathcal{N}(\mu_{t|t-1}^{\mathcal{O}_t, n}, \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t})$$

is sampled. Let

$$\overline{K}_t = \Gamma_{t|t-1}^{\mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1}.$$

The complete component mean is updated by

$$\mu_t^n = \mu_{t|t-1}^n + (\overline{K}_t \otimes I_2) (x_t^{\mathcal{O}_t, n} - \mu_{t|t-1}^{\mathcal{O}_t, n}).$$

The playing-team rows of $\overline{K}_t$ form the identity, so their entries equal the sampled values. The remaining rows produce the conditional means of the non-playing teams. The team-space covariance update is the Schur complement

$$\Gamma_t = \Gamma_{t|t-1} - \Gamma_{t|t-1}^{\mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \Gamma_{t|t-1}^{\mathcal{O}_t},$$

and $\Sigma_t = \Gamma_t \otimes B$. The Kronecker form is preserved by prediction and conditioning. Instead of maintaining a particle-specific $2K \times 2K$ covariance, the algorithm updates one shared $K \times K$ matrix Γ_t , one 2×2 matrix B , and N component means. With dense storage, a covariance update costs $O(K^2)$ after a two-team solve, while all mean updates cost $O(NK)$.

4.5 Rao-Blackwellised SQMC

Rao-Blackwellisation changes the role of SQMC. The full component mean remains $2K$ dimensional, but the stochastic propagation at a match is only four dimensional. The RQMC design therefore has dimension five rather than $2K + 1$.

The Hilbert ordering uses the predicted four-dimensional playing-team means,

$$z_t^n = \text{vec}(\mu_{t|t-1}^{\mathcal{O}_t, n}) \in \mathbb{R}^4.$$

This is a projected ordering. The complete next component is not determined by z_t^n alone because non-playing means remain relevant to later transitions. It should therefore be described as a dimension-reduction heuristic, not as an exact application of a transition-sufficiency theorem. Once an ancestor is selected, its entire $2K$ -dimensional component mean is retained.

4.5.1 Algorithm 3: RB-SQMC match update

Input: component means $\mu_{t-1}^{1:N}$, log-weights, shared covariance, match (i_t, j_t, y_t) and a scrambling key.

1. **Predict.** Apply the OU mean and covariance recursions.
2. **Generate RQMC.** Generate N scrambled Sobol rows $(u_t^n, v_t^n) \in [0, 1]^5$.
3. **Projected Hilbert sort.** Extract the predicted attack-defence means of the two teams, map them to $[0, 1]^4$ and sort their Hilbert keys.
4. **Ordered resampling.** Reorder weights by the Hilbert permutation, sort complete Sobol rows by u_t^n , form the weight CDF and select ancestors.
5. **Retain complete components.** Gather the complete mean of each selected ancestor, including all non-playing teams.
6. **Propagate four coordinates.** If $L_t L_t^\top = \Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \otimes B$, compute

$$x_t^{\mathcal{O}_t, n} = \mu_{t|t-1}^{\mathcal{O}_t, A_n} + L_t \Phi^{-1}(v_t^n).$$

7. **Condition analytically.** Apply $\bar{K}_t \otimes I_2$ to update the complete component mean and use the Schur complement for the shared covariance.
8. **Weight.** Evaluate the bivariate Poisson log-likelihood and normalise with `logsumexp`.
9. **Accumulate evidence.** Add the log average incremental weight to the estimated log normalising constant.

The repository implements the entire operation inside the compiled match scan. The continuous propagation is differentiable. The discrete ancestor selection is stopped in the forward pass, while a differentiable resampling-weight ratio retains the score contribution used by the optimisation code.

4.6 From filtered strengths to match probabilities

For a future match between teams i and j , first propagate each filtered component over the forecast interval. Conditional on a propagated component, the score probabilities are obtained from the bivariate Poisson mass function. The posterior predictive score grid is the mixture

$$p(y_i = a, y_j = b \mid y_{1:t}) \approx \sum_{n=1}^N W_t^n \int G(a, b \mid x^\mathcal{O}, \Theta) p(dx^\mathcal{O} \mid \mu_{t+h|t}^n, \Sigma_{t+h|t}).$$

The inner expectation may be evaluated with the same Gaussian transform used by the filter. Summing cells gives the win, draw and loss probabilities:

$$p(i \text{ wins}) = \sum_{a>b} p(a, b), \quad p(\text{draw}) = \sum_{a=b} p(a, b), \quad p(j \text{ wins}) = \sum_{a<b} p(a, b).$$

Repeatedly updating the filter with historical results produces a time series of posterior attack and defence strengths. Propagating without an observation reverts uncertain teams towards the common long-run distribution, while covariance allows information from one match to affect conditionally related teams. Sequential prediction can then assimilate completed matches and update the probabilities for later fixtures without refitting the model from scratch.

4.7 Parameter estimation

The static parameter vector is

$$\Theta = (\Gamma_0, B, \kappa, \alpha, \beta),$$

subject to positive-definiteness and $\det(B) = 1$. With resampling at each valid match, the normalising-constant estimate is accumulated as

$$\widehat{Z}_t = \widehat{Z}_{t-1} \left[\frac{1}{N} \sum_{n=1}^N G_t(y_t \mid x_t^{\mathcal{O}_t, n}) \right],$$

so

$$\log \widehat{Z}_T = \sum_{t=1}^T \log \left[\frac{1}{N} \sum_{n=1}^N G_t(y_t \mid x_t^{\mathcal{O}_t, n}) \right].$$

The implementation maximises this particle approximation using automatic differentiation. Discrete resampling requires special treatment: differentiable particle-filtering corrections preserve the numerical forward weights while retaining a resampling score in the derivative [Scibior, Masrani and Wood \(2021\)](#). The resulting finite- N objective and gradient remain approximations; optimisation diagnostics should therefore be reported across particle counts and independent scrambles.

5 Unified empirical evaluation

5.1 Statistical and computational questions

The evaluation should connect the method, implementation and application:

1. Does SQMC reduce filtering error relative to SMC at the same N ?
2. Does the GPU offset SQMC’s additional sort and scan costs?
3. Does Rao-Blackwellisation preserve the SQMC advantage as the number of teams grows?
4. Does RB-SQMC improve predictive log score, calibration or particle diversity for football matches?

The implementation comparison should contain SMC-CPU, SMC-GPU, SQMC-CPU and SQMC-GPU where available. Omitting SMC-GPU would confound hardware and method effects. All variants should share the same model, precision, data split and particle counts.

5.2 Accuracy and time-to-solution

On a synthetic model with known latent state, report

$$\text{RMSE}(N) = \left[\frac{1}{TR} \sum_{r=1}^R \sum_{t=1}^T \|\hat{x}_{t,r}^N - x_t^*\|_2^2 \right]^{1/2},$$

where R denotes independent runs or scrambles. The headline combined measure is time-to-accuracy,

$$T_\epsilon = \inf\{t_{\text{wall}} : \text{RMSE}(t_{\text{wall}}) \leq \epsilon\}.$$

This measure rewards both statistical efficiency and fast execution.

5.3 Throughput and kernel timing

JAX execution is asynchronous, so benchmarked calls must finish with `jax.block_until_ready`. Compilation and warm-up should be separated from steady-state timing. Report

$$\text{throughput}(N) = \frac{NT}{t_{N,T}}, \quad S(N) = \frac{t_{\text{CPU}}(N)}{t_{\text{GPU}}(N)}.$$

The kernel breakdown should report RQMC generation, coordinate mapping, Hilbert-key construction, sorting, CDF scan, ancestor search, propagation and likelihood evaluation. Hardware, memory, JAX/JAXLIB version, backend, precision, warm-up policy, repeat count, median and interquartile range belong with every timing result.

5.4 Football predictive evaluation

Use a chronological train-test split so that no future result contributes to a past prediction. For each held-out match, score the full predictive grid using the logarithmic score

$$\text{LogScore}_t = -\log p_t(y_{t,i}, y_{t,j}),$$

and evaluate outcome probabilities with the multiclass Brier score

$$\text{Brier}_t = \sum_{c \in \{W,D,L\}} \{p_t(c) - \mathbf{1}(y_t = c)\}^2.$$

Reliability diagrams should compare predicted and empirical win/draw/loss frequencies. Report predictive performance for RB-SMC and RB-SQMC at matched particle counts and matched wall-clock budgets. The latter is essential because SQMC’s statistical advantage and accelerator throughput are intended to act together.

Particle diversity should be examined before and after resampling. If m_t^n is a vector of component means, define

$$\bar{m}_{\text{pre}} = \sum_{n=1}^N W_t^n m_t^n,$$

$$C_{\text{pre}} = \sum_{n=1}^N W_t^n (m_t^n - \bar{m}_{\text{pre}})(m_t^n - \bar{m}_{\text{pre}})^\top,$$

and compute the analogous equally weighted covariance after resampling. Trace, log determinant and rank diagnostics reveal whether the resampler collapses important directions in the attack-defence state.

5.5 Dimension and long-horizon experiments

For synthetic problems, vary effective dimension and record both accuracy and bits per packed Hilbert coordinate $\lceil 62/d \rceil$. For the football model, compare the four-dimensional projected ordering with a full-state ordering at smaller K where the latter is computationally feasible. This tests the central dimension-reduction choice rather than assuming it is always beneficial.

The long-horizon safety experiment should reproduce the linear Gaussian model analysed by Gerber (2026), track Kolmogorov distance and state clearly that a finite simulation illustrates rather than proves an almost-sure infinite-time result.

No numerical result is asserted in this draft. Expected accuracy or GPU gains must remain hypotheses until the controlled benchmarks have been run.

6 Discussion and conclusion

The complete method is best understood as a chain of compatible reductions. SQMC reduces integration error by replacing independent uniforms with a low-discrepancy design. Hilbert ordering reduces a multidimensional particle cloud to the scalar order required by inverse-transform resampling. JAX reduces the implementation to compiled array primitives that can remain on an accelerator. Finally, Rao-Blackwellisation reduces the football model’s sampled state from $2K$ coordinates to four match-specific coordinates.

None of these reductions is free. Generic SQMC has $O(N \log N)$ sorting cost; the cumulative-weight scan remains a collective operation; a 62-bit packed Hilbert key becomes coarser as dimension grows;

and the four-dimensional football ordering is a projected heuristic because the complete transition retains non-playing component means. These qualifications should remain visible in both the theory and the empirical evaluation.

The football model nevertheless provides a particularly natural application. Its non-Gaussian score likelihood requires particle methods, its linear Gaussian transition permits exact conditional updates, and its sparse two-team observations keep the SQMC propagation dimension small. The result is an online model that can update correlated attack and defence strengths, produce full score probabilities and revise later forecasts as matches are played.

The final criterion is time-to-predictive-accuracy. A successful implementation must show not only that GPU kernels are fast or that SQMC has a favourable asymptotic theory, but that the combined RB-SQMC system reaches a calibrated, accurate football forecast sooner than its SMC alternatives. The experiments defined above are designed to test that claim without conflating statistical, model and hardware effects.

7 References

- Chopin, N. and Gerber, M. (2017). Sequential quasi-Monte Carlo: introduction for non-experts, dimension reduction, application to partly observed diffusion processes. *Monte Carlo and Quasi-Monte Carlo Methods 2016*, 109-139. doi:[10.1007/978-3-319-91436-7_6](https://doi.org/10.1007/978-3-319-91436-7_6).
- Chopin, N. and Papaspiliopoulos, O. (2020). *An Introduction to Sequential Monte Carlo*. Springer. doi:[10.1007/978-3-030-47845-2](https://doi.org/10.1007/978-3-030-47845-2).
- Duffield, S., Power, S. and Rimella, L. (2024). A state-space perspective on modelling and inference for online skill rating. *Journal of the Royal Statistical Society: Series C*. doi:[10.1093/jrsssc/qlae035](https://doi.org/10.1093/jrsssc/qlae035).
- Gerber, M. and Chopin, N. (2015). Sequential quasi-Monte Carlo. *Journal of the Royal Statistical Society: Series B*, 77(3), 509-579. doi:[10.1111/rssb.12084](https://doi.org/10.1111/rssb.12084).
- Gerber, M. (2026). Safety of particle filters: some results on the time evolution of particle filter estimates. *Electronic Journal of Probability*, 31. doi:[10.1214/26-EJP1581](https://doi.org/10.1214/26-EJP1581).
- Karlis, D. and Ntzoufras, I. (2003). Analysis of sports data by using bivariate Poisson models. *The Statistician*, 52(3), 381-393. doi:[10.1111/1467-9884.00366](https://doi.org/10.1111/1467-9884.00366).
- Keller, A., Wachter, C. and Binder, N. (2022). Rendering along the Hilbert curve. arXiv:2207.05415. [arXiv record](https://arxiv.org/abs/2207.05415).
- Keller, A., Wachter, C. and Binder, N. (2023). Quasi-Monte Carlo algorithms (not only) for graphics software. arXiv:2307.15584. [arXiv record](https://arxiv.org/abs/2307.15584).
- Murray, L. M., Lee, A. and Jacob, P. E. (2016). Parallel resampling in the particle filter. *Statistics and Computing*, 26, 789-805. doi:[10.1007/s11222-014-9545-3](https://doi.org/10.1007/s11222-014-9545-3).
- Scibior, A., Masrani, V. and Wood, F. (2021). Differentiable particle filtering without modifying the forward pass. arXiv:2106.10314. [arXiv record](https://arxiv.org/abs/2106.10314).